

**Educação
Profissional
Paulista**

Técnico em
**Ciência de
Dados**

Bibliotecas: Pandas, NumPy, SciPy, Matplotlib e Seaborn

NumPy - Operação de *arrays*

Aula 2

Código da aula: [DADOS]ANO1C2B3S21A2

Bibliotecas:
Pandas, NumPy,
SciPy, Matplotlib e
Seaborn

Mapa da Unidade 5 Componente 3

Introdução ao Python para
análise de dados

semana
18

semana
21

Você está aqui!

NumPy: operação de *arrays*

semana
25

Pandas: transformando
data frame

semana
30

Matplotlib:
gráficos básicos

semana
32

Seaborn: prática

Bibliotecas:
Pandas, NumPy,
SciPy, Matplotlib e
Seaborn

Mapa da Unidade 5 Componente 3

Você está aqui!

NumPy - Operação de *arrays*

Aula 2

21

Código da aula: [DADOS]ANO1C2B3S21A2



Objetivos da aula

- Conhecer as operações básicas entre *arrays* usando a biblioteca NumPy.



Recursos didáticos

- Recurso audiovisual para exibição de vídeos e imagens;
- Acesso ao laboratório de informática e/ou à internet;
- Software Anaconda/Jupyter Notebook ou similar instalado.



Duração da aula

50 minutos.



Competências técnicas

- Ser proficiente em linguagens de programação para manipular e analisar grandes conjuntos de dados;
- Usar técnicas para explorar e analisar dados, aplicar modelos estatísticos, identificar padrões, realizar inferências e tomar decisões baseadas em evidências.



Competências socioemocionais

- Colaborar efetivamente com outros profissionais, como cientistas de dados e engenheiros de dados;
- Trabalhar em equipes multifuncionais colaborando com colegas, gestores e clientes.

Construindo o conceito

Operações

A função **np.sum()** nos permite calcular a soma dos elementos de um *array*.

```
1 a = np.array([1, 2, 3, 4])
2 a.sum()
```

10

```
1 b = np.array([[1, 1], [2, 2]])
```

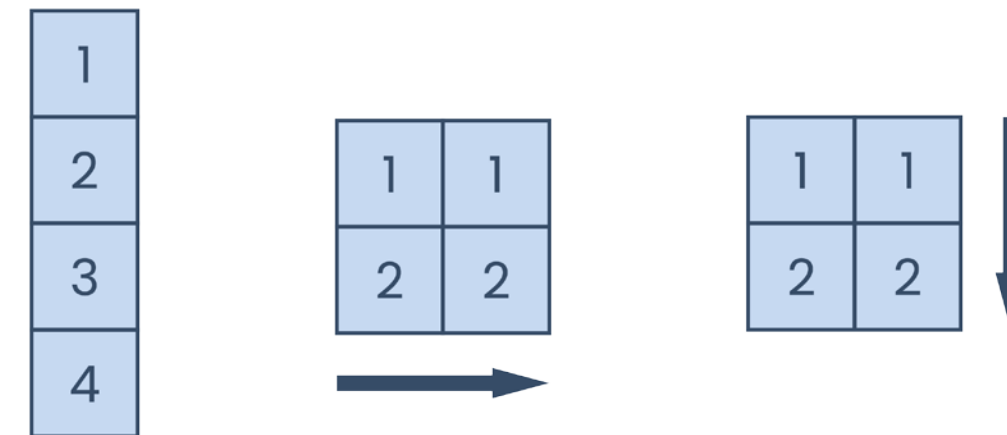
```
1 b.sum(axis=0)
```

array([3, 3])

```
1 b.sum(axis=1)
```

array([2, 4])

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.



`np.sum(axis=0)`: Realiza a soma ao longo do eixo das colunas.

`np.sum(axis=1)`: Realiza a soma ao longo do eixo das linhas.

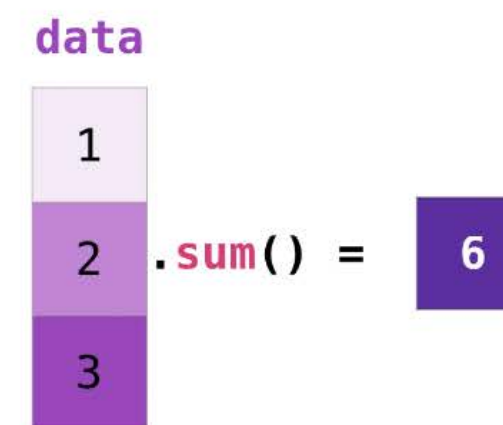
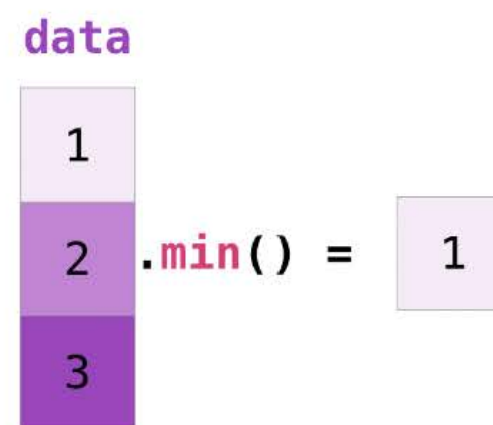
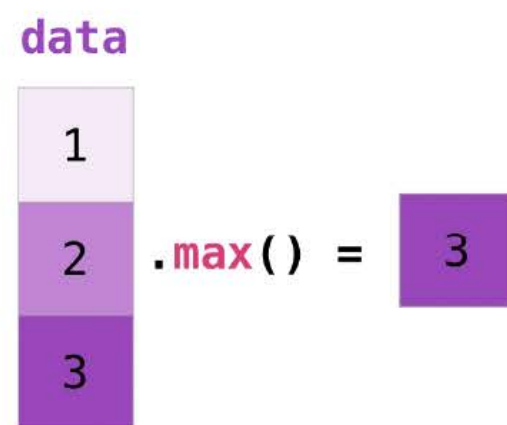
Fonte: NUMPY, [s.d.]a.

Elaborado especialmente para o curso.

Construindo o conceito

Operações

As funções **np.min()** e **np.max()** são utilizadas para encontrar os valores mínimo e máximo, respectivamente, em um *array* NumPy.



Reprodução - NUMPY, [s.d.]a.

Construindo o conceito

Operações

A função **np.min()** retorna o valor mínimo em um *array* NumPy.

Ela aceita o *array* como entrada e pode opcionalmente receber o parâmetro *axis* para especificar ao longo de qual eixo realizar a operação.

Se nenhum eixo for especificado, a função considera todos os elementos do *array* para determinar o valor mínimo.

data		
1	2	.max() =
3	4	
5	6	
		6

data		
1	2	.min() =
3	4	
5	6	
		1

data		
1	2	.sum() =
3	4	
5	6	
		21

Reprodução - NUMPY, [s.d.]a.

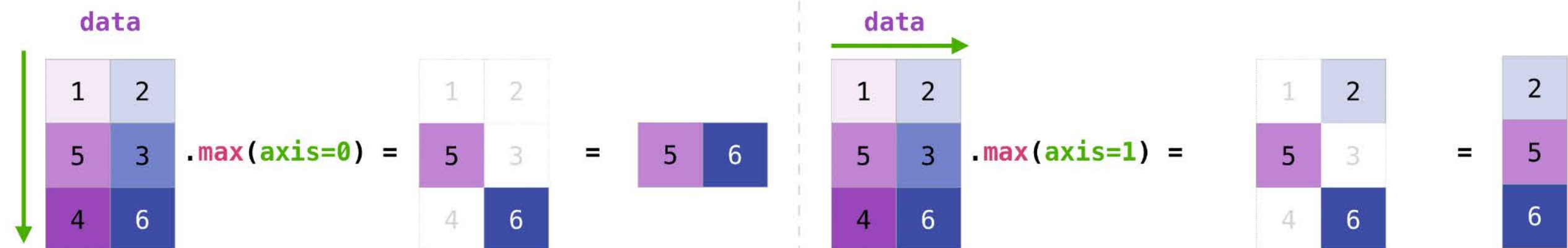
Construindo o conceito

Operações

A função **np.max()** retorna o valor máximo em um *array* NumPy.

Assim como `np.min()`, ela também aceita o *array* como entrada e pode opcionalmente receber o parâmetro *axis* para especificar ao longo de qual eixo realizar a operação.

Se nenhum eixo for especificado, a função considera todos os elementos do *array* para determinar o valor máximo.



Reprodução - NUMPY, [s.d.]a.

Operações

1. Soma de um escalar a um vetor:

- ▶ Quando você soma um escalar (um número) a um vetor, o escalar é adicionado a cada elemento do vetor.
- ▶ Por exemplo, se você tem o vetor $[1, 2, 3]$ e deseja adicionar o escalar 5, o resultado será $[6, 7, 8]$.
- ▶ Essa operação é realizada elemento a elemento, independentemente do tamanho do vetor.

```
a
```

```
array([1, 2, 3, 4])
```

```
a + 1
```

```
array([2, 3, 4, 5])
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Operações

2. Soma de um escalar a uma matriz:

Se você somar um escalar a uma matriz, o escalar será adicionado a cada elemento da matriz.

```
b
```

```
array([[1, 1],  
       [2, 2]])
```

```
b + 1
```

```
array([[2, 2],  
       [3, 3]])
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o conceito

Operações

Há momentos em que você pode querer realizar uma **operação entre um *array* e um único número** (também chamada de operação entre **um vetor e um escalar**) ou entre *arrays* de dois tamanhos diferentes.

Por exemplo, uma matriz (vamos chamá-la de “array”) pode conter informações sobre distância em milhas, mas você deseja converter as informações em quilômetros. **Você pode realizar essa operação com:**

$$\begin{array}{|c|} \hline 1 \\ \hline 2 \\ \hline \end{array} * 1.6 = \begin{array}{|c|} \hline 1 \\ \hline 2 \\ \hline \end{array} * \begin{array}{|c|} \hline 1.6 \\ \hline 1.6 \\ \hline \end{array} = \begin{array}{|c|} \hline 1.6 \\ \hline 3.2 \\ \hline \end{array}$$

Fonte: NUMPY, [s.d.]a.

```
array1 = np.array([1.0, 2.0])  
array1 * 1.6
```

```
array([1.6, 3.2])
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o conceito

Operações

O que acontece quando somamos *arrays* e eles não têm a mesma dimensão?

$$\begin{array}{c} \text{array 1} \\ \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \end{array} + \begin{array}{c} \text{array 2} \\ \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \end{array} = \begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix}$$

$$\begin{array}{c} \text{array 1} \\ \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \end{array} + \begin{array}{c} \text{array_linha} \\ \begin{bmatrix} 1 & 1 \end{bmatrix} \end{array} = \begin{array}{c} \text{array 1} \\ \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \end{array} + \begin{array}{c} \text{array_linha} \\ \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{bmatrix} \end{array} = \begin{bmatrix} 2 & 3 \\ 4 & 5 \\ 6 & 7 \end{bmatrix}$$

Fonte: NUMPY, [s.d.]a.
Elaborado especialmente para o curso.

Construindo o conceito

Operações

Você pode fazer essas **operações aritméticas** em matrizes de tamanhos diferentes, **mas somente se uma matriz tiver apenas uma coluna ou linha**.

Nesse caso, o NumPy utilizará suas regras de transmissão para a operação.

```
array1 = np.array([[1, 2], [3, 4]])  
array2 = np.array([[1, 1], [1, 1]])  
resultado1 = array1 + array2  
resultado1
```

```
array([[2, 3],  
       [4, 5]])
```

```
array1 = np.array([[1, 2], [3, 4], [5, 6]])  
array_linha = np.array([[1, 1]])
```

```
resultado = array1 + array_linha  
resultado
```

```
array([[2, 3],  
       [4, 5],  
       [6, 7]])
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o conceito

Operações

Operador * (multiplicação elementar):

O operador * realiza a multiplicação elemento a elemento nos elementos dos *arrays*. Para cada elemento na posição (i, j) , o valor resultante é obtido multiplicando-se o elemento correspondente em $a[i][j]$ pelo elemento correspondente em $b[i][j]$.

```
A = np.array([[1, 1],  
              [0, 1]])  
B = np.array([[2, 0],  
              [3, 4]])
```

A

```
array([[1, 1],  
       [0, 1]])
```

B

```
array([[2, 0],  
       [3, 4]])
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Operações

Operador @ (multiplicação de matrizes):

O operador @ (ou `np.matmul()`) realiza a multiplicação de matrizes. Ele verifica se o número de colunas da primeira matriz é igual ao número de linhas da segunda matriz antes de executar a multiplicação.

Função `dot()`:

A função `dot()` também realiza a multiplicação de matrizes. É equivalente ao uso do operador @.

```
A * B      # produto elemento a elemento
```

```
array([[2, 0],  
       [0, 4]])
```

```
A @ B      # produto matricial
```

```
array([[5, 4],  
       [3, 4]])
```

```
A.dot(B)   # outro produto matricial
```

```
array([[5, 4],  
       [3, 4]])
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.



Vamos
fazer um
quiz

O que a função `np.dot()` faz em *arrays* NumPy?

Realiza a multiplicação elemento a elemento entre dois *arrays*.

Calcula o produto escalar entre dois *arrays*.

Calcula a soma de produtos ao longo dos eixos especificados.

Retorna o número de dimensões de um *array*.



Vamos
fazer um
quiz

**Qual operação é realizada pelo
operador $+$ em *arrays* NumPy?**

Multiplicação
elemento a elemento.

Soma dos elementos
dos *arrays*.

Produto escalar entre
os *arrays*.

Divisão dos elementos
dos *arrays*.



Vamos
fazer um
quiz

**Que função do NumPy é usada
para calcular a diferença entre
dois *arrays* NumPy?**

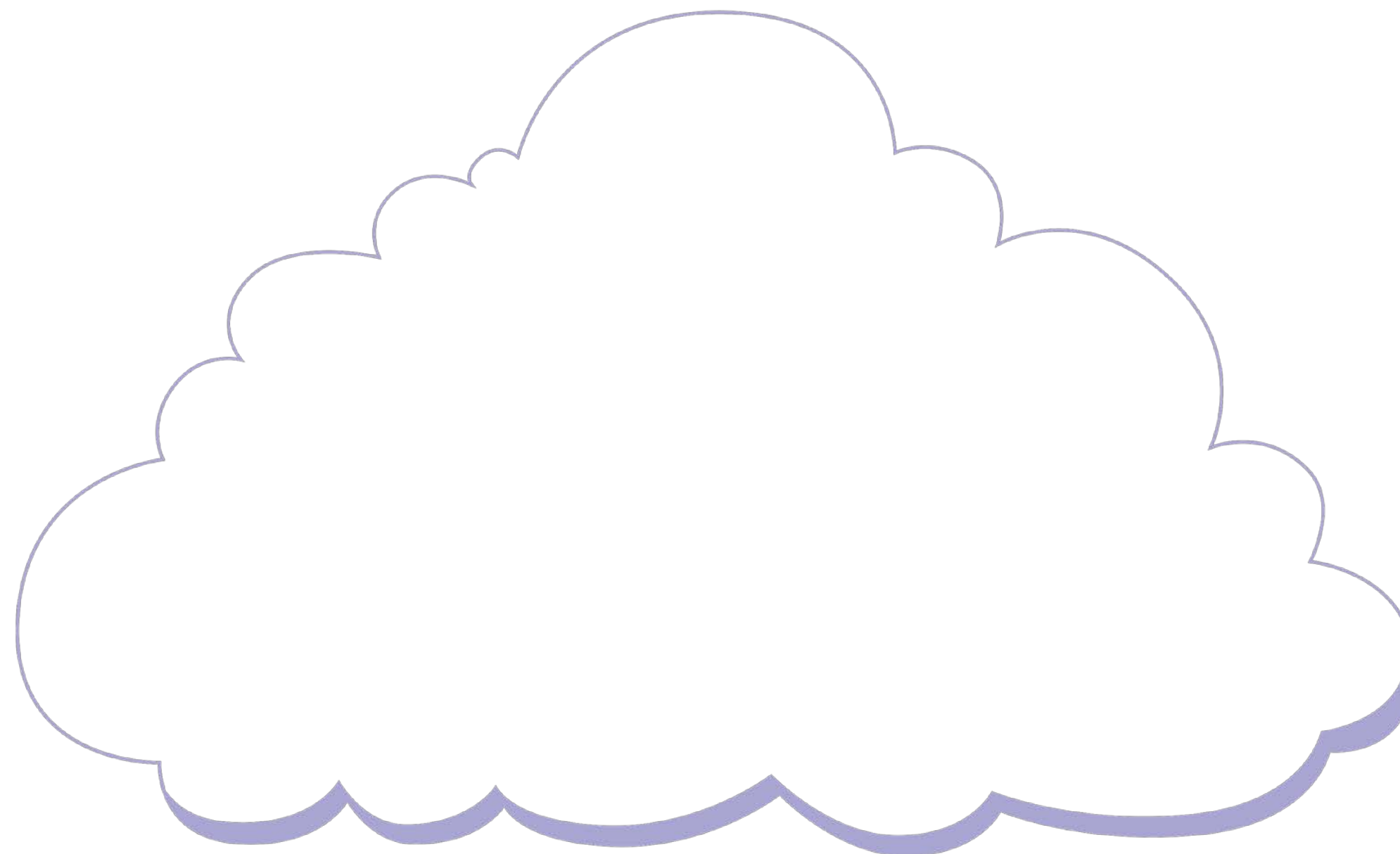
`np.subtract()`

`np.add()`

`np.multiply()`

`np.divide()`

Nuvem de palavras



© Getty Images

O que nós
aprendemos
hoje?



© Getty Images

O que nós
**aprendemos
hoje?**

Então ficamos assim...

- 1** A multiplicação matricial, elemento a elemento, é diferente do produto matricial;
- 2** Vimos que podemos fazer operações entre *arrays* e escalar;
- 3** Conhecemos mais algumas funções do NumPy.

Saiba mais

**Ainda tem dúvida sobre o NumPy?
Vale a pena ler este artigo!**

ALMEIDA, M. Numpy: trabalhando computação científica com Python. *Alura*, 13 mar. 2023.

Disponível em:

<https://www.alura.com.br/artigos/numpy-computacao-cientifica-com-python>.

Acesso em: 28 maio 2024.

Referências da aula

Identidade visual: imagens © Getty Images

MCKINNEY, W. *Python para análise de dados: tratamento de dados com Pandas, NumPy & Jupyter*. São Paulo: Novatec, 2023.

NUMPY: the absolute basics for beginners. *NumPy*, [s.d.]a. Disponível em: https://numpy.org/doc/stable/user/absolute_beginners.html. Acesso em: 28 maio 2024.

PÁGINA inicial. *NumPy*, [s.d.]b. Disponível em: <https://numpy.org/pt/>. Acesso em: 28 maio 2024.

Educação Profissional Paulista

Técnico em
**Ciência de
Dados**